WEEKLY PROGRESS REPORT -05

SUPABASE BACKEND, DATABASE SCHEMA & SUPABASE AUTHENTICATION (LOGIN / SIGNUP)

Project:	MuktoKormo (Freelance Marketplace)	**Scope:**	Database & Authentication Wiring
Frontend:	Next.js 16, React 19, Tailwind v4	**Backend:**	**Supabase (PostgreSQL, Auth, RLS)**
Sprint:	Week 5 (Supabase Integration)	**Next Phase:**	Live Data Wiring & Session Layer

Team Members: Abdul Ahad Akid (2207018) | Ashraful Hosen (2207042)

1. Project Overview & Architectural Shift

Executive Context: MuktoKormo is an Upwork/Fiverr-grade freelance marketplace engineered for the Bangladeshi digital ecosystem, connecting local employers with verified remote talent through a milestone-based escrow workflow. Earlier sprints delivered the full front-end experience - landing, auth and role-based dashboard flows - on top of local mock services.

Sprint Objectives: This sprint pulled the platform off its mock foundation and connected it to a real hosted Supabase backend: a production PostgreSQL schema deployed from the platform specification's ER diagram, Row Level Security enforced on every table, and both authentication surfaces (/login and /signup) wired to Supabase Auth.

Architectural Status: The presentation layer (Next.js 16 / React 19 / Tailwind v4) is unchanged. The backend is now Supabase - managed PostgreSQL with PostgREST auto-APIs, GoTrue authentication (v2.197), and per-row RLS. The hosted project is live (ref dhkhwaazyzikaoaontzn); the new-format publishable API key was validated at runtime against the auth and PostgREST endpoints.

2. Supabase Provisioning & Environment Setup

- A hosted Supabase project was provisioned. The new-format *publishable* API key is stored in **.env.local** (git-ignored, never committed) with a committable template in **.env.example**.
- Exactly two runtime dependencies were added - **@supabase/supabase-js** (2.117.2) and **@supabase/ssr** - keeping the toolchain minimal and the Next.js bundle footprint small.
- Two thin client factories were created: **src/lib/supabase/client.ts** (browser, createBrowserClient) and **src/lib/supabase/server.ts** (Server Components / route handlers, using the async cookies() API of this Next.js version). Both expose the session and auth helpers used by the login and signup routes.
- Environment validated at runtime before any schema work: the API key decodes and matches the project ref, /auth/v1/health returns 200, and PostgREST answers requests - rejecting only genuinely absent tables, which proves the key is accepted and the project is online.

3. Database Schema Deployment (from the Specification ER Diagram)

The diagram from the platform specification was reproduced with a single versioned migration, **supabase/migrations/20261007000000_initial_schema.sql** (48 statements), applied to the hosted database and independently validated with a PostgreSQL grammar (pglast) before execution. It creates the seven core tables exactly as diagrammed plus supporting indexes, constraints, and the signup trigger:

Table	Purpose	Key Columns	Relations & Integrity
users	Identity directory (1:1 with auth.users)	id (uuid PK), email UNIQUE, name, created_at	id FK -> auth.users.id; row created by handle_new_user trigger
clients	Employer / client company profile	id (PK), user_id, company_name, location	user_id FK UNIQUE -> users.id (1:1)
freelancers	Freelancer professional profile	id (PK), user_id, headline, bio, skills text[], hourly_rate numeric(12,2)	user_id FK UNIQUE -> users.id (1:1)
projects	Job postings created by clients	id (PK), client_id, title, description, category, budget_min, budget_max, timeline_days, status	client_id FK -> clients.id (1:N); status CHECK (draft/open/in_progress/completed/archived); budget & date checks
proposals	Freelancer offers on a project	id (PK), project_id, freelancer_id, cover_letter, proposed_amount, timeline_days, status	project_id FK -> projects.id (1:N); freelancer_id FK; UNIQUE (project_id, freelancer_id); status CHECK
contracts	Agreed engagement (escrow workflow)	id (PK), proposal_id, total_amount, milestone_count, status, signed_at	proposal_id FK UNIQUE -> proposals.id (1:1); status CHECK (pending/active/completed/cancelled)
reviews	Post-contract rating & feedback	id (PK), contract_id, reviewer_id, reviewee_id, rating, comment	contract_id FK UNIQUE -> contracts.id (1:1); rating CHECK 1-5; reviewer/reviewee FKs

Design notes: money is stored as numeric(12,2) to avoid float drift on escrow math; every status field carries a CHECK constraint; all foreign keys are indexed; the 1:1 relationships (proposals->contracts, contracts->reviews, users->profiles) are enforced with UNIQUE foreign keys so no application can create duplicates. The password column present in the diagram's USERS box is intentionally not re-invented: passwords are owned by Supabase Auth and stored as bcrypt hashes in auth.users.encrypted_password - never plaintext, never exposed through the API. public.users.id references auth.users.id, and a handle_new_user trigger materialises users (email + full name) on every signup.

4. Security: Row Level Security (RLS)

RLS is enabled on all seven tables and was proven live. Anonymous reads return zero rows on every table and anonymous writes are rejected with PostgreSQL error **42501 (new row violates row-level security policy)**.

The permission matrix, aligned with the security requirements of the platform report, is:

Role	Available on	Permitted operations
anon	All seven tables	None. Reads return zero rows; writes rejected (42501 - row-level security violation). Proves tables are fully closed.
authenticated - users	users	select / insert / update own row only where auth.uid() = id.
authenticated - clients	clients / projects	Directory reads of client profiles; project create / update / delete for own projects (client_id = own row).
authenticated - freelancers	freelancers / proposals	Directory reads of freelancer profiles; create / update own proposals; withdraw own proposals.
authenticated - party-scoped	contracts / reviews	Access resolved through joins to auth.uid() - project owner (client) or proposal owner (freelancer); only parties may act.

The anon role receives nothing - every table is closed by default. All write policies bind to auth.uid(), so a user can only ever touch rows they own or rows explicitly addressed to them as a contract party.

5. Authentication - Login (/login)

- Credentials are verified server-side by **supabase.auth.signInWithPassword**; the returned session is persisted via @supabase/ssr cookie storage, so it survives page reloads.
- Role-aware redirect preserved from the earlier sprint: after sign-in the user is routed to the matched Client or Freelancer dashboard based on the profile row that exists for them.
- Password reset is wired through **resetPasswordForEmail**, with an in-page confirmation note and a visible error banner for failed attempts; the Forgot? link is kept on the credential form.
- SMS-OTP and Google/Apple affordances were removed this week: they require external provider credentials (SMS gateway, Google and Apple developer consoles) that are outside the current scope. The surface is simplified to the email & password form, with OAuth/OTP tracked as future work.

6. Authentication - Signup (/signup)

- Both role flows call **supabase.auth.signUp** with raw_user_meta_data.full_name; the database trigger materialises users.name automatically, so no extra round-trip is needed.
- Freelancer flow: the 4-step wizard (role, talent profile, payment, submit) finishes with signUp and then inserts the freelancers profile row - headline, hourly rate, skills list, bio - under the fresh session.
- Client flow: a new "Your account" step (full name / email / password / terms acceptance) was added to the wizard, followed by signUp and creation of the clients profile row (company name).
- Email-confirmation aware: with the "Confirm email" toggle enabled the app presents a confirmation screen after signup; with auto-confirm it redirects straight into the dashboard. The same code path supports both modes.
- Passwords are never handled by application code - signUp posts to GoTrue, which hashes with bcrypt before storing in auth.users.

7. End-to-End Verification

- A disposable signup succeeded end-to-end: the auth user was created, the trigger auto-created the public.users row with the correct email and name, a session was returned, own-row reads/updates worked, and a self-scoped clients insert was accepted.
- Cross-tenant reads were suppressed - the new user could see only their own users row, confirming the RLS policies behave as designed from the very first signup.
- The anonymous write probe returned 42501 on all seven tables, proving the tables are closed to public writes.
- The migration file was syntax-checked against a production PostgreSQL grammar (pglast) before execution, and every constraint and index was confirmed present on the live database.
- All credentials verified: project ref matches the key binder, health endpoint 200, PostgREST reachable, and the signup/login flows complete with the real anon key.

8. Quality Assurance

- TypeScript strict check (tsc --noEmit): clean across all new auth and Supabase files.
- Production build (next build): every route compiles and prerenders without runtime errors.
- ESLint: the 21 pre-existing findings are confined to legacy modal / footer / dashboard components from earlier sprints; none of the new Supabase or auth code adds findings.
- No regressions in existing public routes, dashboards, or the post-job flow.

9. Work in Progress & Next Sprint Objectives

- Add middleware (proxy.ts) for server-side session refresh and a sign-out control using @supabase/ssr.
- Replace the dashboard, feed, and job-post mock datasets with live Supabase queries behind the existing component contracts - no visual regressions, real data underneath.
- Introduce a role/admin concept, live project and proposal creation, plus dispute and messaging persistence on the deployed tables.
- Provision Storage buckets for KYC and portfolio files, and enable Realtime for proposals and in-chat messages.
- Re-introduce SMS-OTP login (requires an SMS provider account) and Google/Apple OAuth (requires provider console configuration) once the credentials are available.

Deliverables this sprint: live Supabase project wired via .env.local & thin clients; 7-table schema migration with constraints, indexes and RLS; handle_new_user trigger; /login on signInWithPassword + resetPasswordForEmail; /signup on signUp with profile creation for both roles; client-account wizard step; end-to-end verification against the hosted project. Weekly report generated from the sprint record.